Revision 32–36 · Batching and scheduling: static, per-step, continuous, budgets & preemption

Interactive revision of lesson 32, lesson 33, lesson 34, lesson 35 and lesson 36: one question at a time, logged in my own reasoning.

Q1 · Static batching bill PARTIAL

Covers: 32, 33

QUESTION

A static batch of 4 slots gets replies of 50, 150, 400 and 800 tokens at ~5 ms per step. What share of slot-steps does useful work, when does the 50-token user get their answer, and what changes if the scheduler decides after every step?

How I solved it

1. A static batch runs until its longest reply ends: **800 steps**, so capacity = 4 slots × 800 = **3,200 slot-steps**.
2. Useful work = tokens actually generated: 50 + 150 + 400 + 800 = **1,400**.
3. Useful share = 1,400 ÷ 3,200 ≈ **44%**; the other 56% is padding.
4. Static: the 50-token answer is handed back with the batch, after 800 × 5 ms = **4 s**.
5. Per-step scheduling: it leaves after 50 × 5 ms = **0.25 s** (16× sooner), and its slot goes to a waiting request on the next step.

RE-DRILL

Re-drill: compute the useful share for any length mix in one line, and say what happens to it as the batch grows.

Q2 · When is static batching fine? SOLID

Covers: 32

QUESTION

A nightly job classifies 10 million reviews with exactly one output token each; a live chatbot's replies range from 20 to 1,500 tokens. Which can use static batching without much loss, and why?

Answer

- Static batching is fine when outputs are **uniform**: the nightly job's useful share is mean ÷ longest = 1 ÷ 1 ≈ **100%**, so there's almost no padding.
- The chatbot's lengths vary a lot: if the average reply is a few hundred tokens and the longest in a batch is ~1,500, the share is around **20%**, and short answers wait for long ones. It needs continuous batching.
- Caveat: the nightly job's **prompts** still differ in length, so a static batch pads them to the longest prompt at prefill; sorting reviews by length before batching mostly removes that.

> **RE-DRILL**
>
> Re-drill: put a number on it with mean ÷ longest instead of stopping at "uniform vs varied".

Q3 · Ragged step PARTIAL

Covers: 33, 34

> **QUESTION**
>
> On one step two requests decode (one token each) and a new request brings a 4-token prompt. A junior suggests padding all three to 4 tokens. What fraction is wasted, how does Orca's selective batching avoid padding, and which layers can and can't run on all tokens together?

Answer

- Padded: 3 rows × 4 = **12 slots** for 1 + 1 + 4 = **6 real tokens**, so **50% wasted**.
- Selective batching flattens the 6 tokens into one stack (no rectangle needed) and runs the **token-wise layers once on all of them**: QKV and output projections, MLP, norms, residual adds.
- **Attention** is split per request and merged back afterwards: each token may only look at earlier tokens **from its own request**, in that request's KV cache, and every request has a different history length.
- Today's engines do the same with a ragged batch plus an attention mask (or per-request offsets in the attention kernel).

> **RE-DRILL**
>
> Re-drill: count real tokens vs padded slots for any mix, and name every token-wise layer in a block.

Q4 · The scheduler is now on the clock PARTIAL

Covers: 31, 33

> **QUESTION**
>
> With iteration-level scheduling each GPU step takes 5 ms and the Python scheduler spends 2 ms per step deciding who runs, with the GPU idle meanwhile. What fraction of GPU time is wasted, what happens to throughput, and why was this overhead invisible under request-level scheduling?

How I solved it

1. The scheduler and the GPU run one after the other, so one cycle = 5 + 2 = **7 ms**.
2. Wasted fraction = idle ÷ cycle = 2 ÷ 7 ≈ **29%**.
3. Each cycle still yields one token per running request, so throughput ∝ 1 ÷ cycle time: (1/7) ÷ (1/5) = 5 ÷ 7 ≈ **71%** of the zero-overhead throughput.
4. Request-level scheduling decided once per batch, every few seconds (e.g. 1,000 steps × 5 ms = 5 s), so 2 ms was 2 ÷ 5,000 ≈ **0.04%**. Deciding every step multiplies the overhead by the number of steps.
5. Fixes: keep the scheduler's work tiny, and **overlap** it with the GPU (prepare step N+1 while step N runs; vLLM's async scheduling), plus CUDA graphs to cut launch cost.

> **RE-DRILL**
>
> Re-drill: for serial overheads, always add them to the cycle before dividing; check that wasted % + kept % = 100%.

Q5 · Who goes first? SOLID

Covers: 34, 35

> **QUESTION**
>
> Continuous batching schedules every running decode first, then admits new prompts into the leftover budget. A teammate proposes prompts first "to cut TTFT for new users". What happens to users mid-answer, and why do engines put decodes first?

Answer

- Prompts first means big prefills fill the step: the step gets long and users mid-answer **stall** (inter-token latency spikes). If prompts use up the whole token budget, running requests can be **skipped entirely** for that step.
- Admitting prompts first can also grab KV memory that running requests need to grow, forcing **evictions** of work already half done.
- Decodes first is cheap: each running request costs **one token**, and below the ~300-token knee those tokens are nearly free, so streams stay smooth and requests **finish and free their slots and KV** sooner.
- TTFT for new users is protected another way: leftover budget plus **chunked prefill**, so a long prompt progresses every step without stalling anyone.

RE-DRILL

Re-drill: name both costs of prompts-first: longer steps (ITL) and KV pressure (evictions).

Q6 · Coding: static vs continuous simulator SOLID

Covers: 32, 34

QUESTION

Write `simulate(lengths, slots)` returning total steps, useful share and average finish for static batching (groups of `slots`, each as long as its longest member) and continuous batching (a freed slot goes to the next queued request at once). It must reproduce lesson 34's 12-request example.

How I solved it

1. Static: each group runs `max(group)` steps, and every member finishes at `t + max(group)`, so I added one finish time per request.
2. Continuous: keep `free_at` per slot; each request takes the slot that frees first, so `free_at[i] += n` is both its finish and the slot's next free time.
3. Total continuous time is when the busiest slot finishes: `max(free_at)`.
4. Results match: 12 requests on 4 slots give static **(2,200, 48%, 1,633)** vs continuous **(1,150, 92%, 635)**.
5. The second test ([50, 150, 400, 800], 2 slots) has the **same total (950)** both ways, yet average finish drops **550 → 400**: continuous batching helps latency even when the longest request dominates total time.

```
def static(lengths, slots):
    t, finishes = 0, []
    for g in range(0, len(lengths), slots):
        group = lengths[g:g + slots]
        group_steps = max(group)                        # padding: short rows ride along
        finishes += [t + group_steps] * len(group)      # everyone returns with the batch
        t += group_steps
    return t, finishes


def continuous(lengths, slots):
    free_at, finishes = [0] * slots, []
    for n in lengths:
        i = free_at.index(min(free_at))   # FCFS: take the slot that frees first
        free_at[i] = free_at[i] + n       # start when it frees, run n steps
        finishes.append(free_at[i])       # handed back the moment it's done
    return max(free_at), finishes
```

RE-DRILL

Re-drill: explain why the second test's totals are equal (the 800-token request alone sets the end) while average finish still improves.

Q7 · Pick the budget PARTIAL

Covers: 32, 35

QUESTION

Llama 3.1 8B on an H100 (idealized: ~4.8 ms floor, then ~16 μs per token). The product wants a token at least every 25 ms, with 150 users mid-answer on every step. What `max_num_batched_tokens` do you start from, how much is left for prompts, and how many steps does a 6,000-token prompt take?

How I solved it

1. Step time ≈ tokens × 16 μs, and every running user waits one step per token, so step time ≤ 25 ms.
2. Budget = 25 ms ÷ 16 μs = $25 \times 10^{-3} \div 16 \times 10^{-6}$ = 25,000 ÷ 16 ≈ **1,560 tokens**.
3. Decodes go first: 1,560 − 150 = **1,410 tokens** per step for prompts.
4. Prefill steps = 6,000 ÷ 1,410 ≈ 4.3, so **5 steps** (4 full chunks of 1,410, then 360).
5. At ~25 ms per step, that prompt's TTFT is about 5 × 25 ms ≈ **125 ms** of steps.

Caveats: `max_num_seqs` (≥ 150 here) must stay ≤ the budget, and real steps are slower than the idealized model, so treat 1,560 as a starting point and confirm with a benchmark.

RE-DRILL

Re-drill: redo it for a 50 ms target and 300 decoders, and check units before dividing.

Q8 · Which gate is closed? SOLID

Covers: 30, 35

QUESTION

`num_requests_running` is stuck at exactly 64 for an hour, `num_requests_waiting` climbs past 200, KV cache usage is 40%, and steps are well under the latency target. Which admission gate is blocking new requests, and what do you change?

Answer

- Running pinned at a round number while KV and step time have headroom means the **sequence-slot gate** is closed: `max_num_seqs = 64`.
- The other two gates are open: the token budget isn't binding (steps are fast) and KV memory isn't (40% used).
- Fix: **raise `max_num_seqs`** in steps (e.g. 64 → 128), watching KV usage, preemptions and inter-token latency after each change; keep it ≤ `max_num_batched_tokens`.
- KV at 40% doesn't mean 2.5× more requests fit: running requests' caches keep growing as answers get longer, so leave headroom.

RE-DRILL

Re-drill: for each gate, name the metric that shows it binding (running = max_num_seqs; step time near target; KV usage near 100% plus preemptions).

Q9 · Fair across customers PARTIAL

Covers: 36

QUESTION

Client A sends 90% of traffic; small client B complains about terrible p99 TTFT. The PM proposes a hard per-client limit of N requests per minute. What does that cost, what would you propose instead and what exactly does it count, and what happens to A when B goes quiet?

Answer

- Cost of the rate limit: it **rejects A's work even when the GPU has spare room**. In the VTC paper (OSDI '24), strict limits cut throughput from **~780 to ~340 tokens/s**.
- Instead: the **Virtual Token Counter**. Keep a counter per client of the **tokens it has been served, input and output weighted**; whenever there's room, admit the next request from the client with the **lowest counter**.
- B's counter stays low, so B's requests get in **right away** instead of waiting behind a wall of A's.
- When B is quiet, A is the only client with work, so **A can use the whole GPU**: nothing sits idle.
- A client returning from idle has its counter **lifted to the lowest active counter**, so it can't bank idle time as credit.

RE-DRILL

Re-drill: answer every part of a multi-part question, starting with the cost of the naive fix.

Q10 · Preemption storm PARTIAL

Covers: 01, 35, 36

QUESTION

`vllm:num_preemptions_total` climbs every minute and p99 latency spikes; requests average ~2,000 tokens on Llama 3.1 8B. How much KV does one request hold, who gets evicted and how, what does each way cost, and what's the real fix?

Answer

- KV per token = 2 × 32 layers × 8 KV heads × 128 dims × 2 bytes = 131,072 B = **128 KiB**; × 2,000 tokens ≈ **262 MB** per request.
- Victim: under FCFS, the request that **arrived last**.

- **Recompute**: throw its KV away; later re-run prompt + generated tokens as a fresh prefill. Costs **GPU math**: idealized 2,000 × 16 μs ≈ **32 ms**, worse with attention as requests get longer. vLLM V1's default.
- **Swap**: copy the KV to **CPU memory** and back when there's room. Costs **PCIe transfer** (262 MB each way, ≈ 5 ms at ~50 GB/s in theory, slower with many small blocks) plus 262 MB of CPU RAM held.
- Short victims are cheap to recompute; evicting long ones costs a lot either way.
- Real fix: constant preemption is a **sizing problem**. Give KV more memory (higher `gpu_memory_utilization`, quantized weights or KV, more GPUs), or **admit fewer at once** (lower `max_num_seqs`).

> **RE-DRILL**
>
> Re-drill: keep "queue order" (who waits) and "preemption" (who's pushed out of memory) as separate questions.

Q11 · Cancellation PARTIAL

Covers: 36

> **QUESTION**
>
> A user closes the tab at token 300 of a reply capped at `max_tokens = 1,000`, and the server never notices. How many tokens are wasted, how much KV does the request end up holding, and what should the server do instead?

Answer

- The engine keeps generating until the cap: 1,000 − 300 = **700 tokens for nobody**, each costing a slot in every step.
- KV keeps growing to 1,000 × 128 KiB ≈ **131 MB** (Llama 3.1 8B), held until the request ends.
- Instead: **watch the connection**. On disconnect or timeout, **abort** the request; on the very next step its slot and its ~39 MB of KV go to requests still wanted.
- Same principle for **stop sequences and length limits**: stop the moment you know, not at the end.

> **RE-DRILL**
>
> Re-drill: read the scenario's condition ("never notices") before answering with the fix.